MONGODB FUNDAMENTALS AND CRUD OPERATIONS

Student Name: Vidula Saravanan

Topic: MongoDB database, collections, CRUD operations, query operators and a Library Management System use case

Database examples: studentDB and libraryDB

Submission note: This report is prepared from the MongoDB commands and terminal outputs shared during the practical. Clearly marked query-operator sections identify results that still need to be verified in the MongoDB Shell.

Prepared for practical coursework

1. Introduction to MongoDB

MongoDB is a NoSQL, document-oriented database. It stores information as flexible, JSON-like documents using BSON (Binary JSON). Instead of organising information into relational tables and rows, MongoDB groups documents into collections. Each document can contain fields such as strings, numbers, Boolean values and nested data.

MongoDB is commonly used in web applications because its document structure can represent application data in a flexible way. In this assignment, MongoDB is used to manage student records and book records in a small library system.

1.1 Basic MongoDB structure

MongoDB term	Description	Example in this assignment
Database	A container that holds collections.	studentDB, libraryDB
Collection	A group of related documents; similar in purpose to a table in a relational database.	students, books
Document	A single record stored as field-value pairs.	A student record or a book record
Field	A named piece of information inside a document.	name, age, course, status
_id	A unique identifier automatically added to a document when one is not supplied.	ObjectId(...)

2. Database and Collection Setup

The following commands switch to the studentDB database and create a collection named students.

Create or switch to the database

```
use studentDB
```

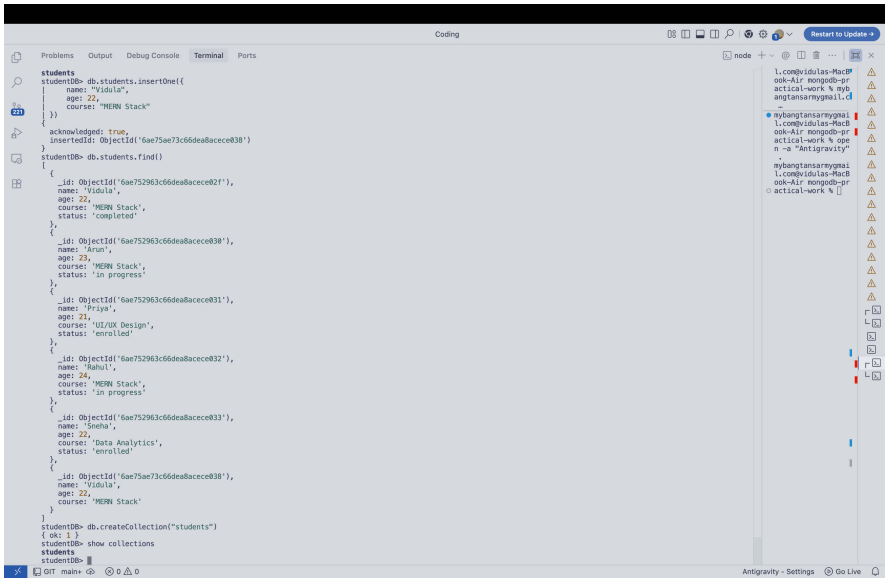
Create the collection

```
db.createCollection("students")
```

Display collections in the current database

```
show collections
```

The collection used for the practical was named **students**. MongoDB Shell switches to a database with the use command; the database is persisted when data is stored in it.



3. Insert Operations (Create)

Insert operations add new documents to a collection. MongoDB provides insertOne() for one document and insertMany() for multiple documents.

3.1 Insert one document

```
db.students.insertOne({
    name: "Vidula",
    age: 22,
    course: "MERN Stack",
    status: "enrolled"
})
```

This command inserts one student document with the fields name, age, course and status. The student record was subsequently updated to completed during the practical.

3.2 Insert multiple documents

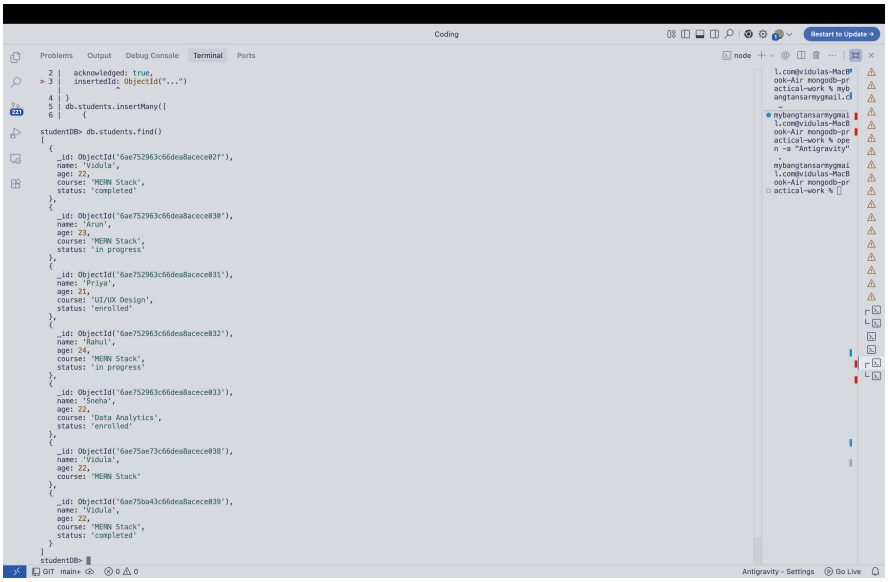
```
db.students.insertMany([
    {
        name: "Arun",
        age: 23,
        course: "MERN Stack",
        status: "enrolled"
    },
    {
        name: "Priya",
        age: 21,
        course: "UI/UX Design",
        status: "completed"
    },
    {
        name: "Rahul",
        age: 24,
        course: "MERN Stack",
        status: "enrolled"
    },
    {
        name: "Sneha",
        age: 22,
        course: "Full Stack Development",
        status: "completed"
    }
])
```

The multiple-document insert operation added student records to the students collection. The records were later updated and deleted during the CRUD practice. A further set of five records was inserted after the collection was cleared for query-operator practice.

3.3 Query-practice dataset

```
db.students.insertMany([
    { name: "Vidula", age: 22, course: "MERN Stack", status: "completed" },
    { name: "Arun", age: 23, course: "MERN Stack", status: "in progress" },
    { name: "Priya", age: 21, course: "UI/UX Design", status: "enrolled" },
    { name: "Rahul", age: 24, course: "MERN Stack", status: "in progress" },
    { name: "Sneha", age: 22, course: "Data Analytics", status: "enrolled" }
])
```

MongoDB confirmed that five documents were inserted successfully. The returned insertedIds showed that each document received its own ObjectId.


Coding
Restart to Update
Problems
Output
Debug Console
Terminal
Ports
node
acknowledged: true,
insertedIds: ObjectId("...")
db.students.insertMany([
studentDB> db.students.find()
_id: ObjectId('6ae752963c66dea8acece02f'),
name: 'Vidula',
age: 22,
course: 'MERN Stack',
status: 'completed'
_id: ObjectId('6ae752963c66dea8acece030'),
name: 'Arun',
age: 23,
course: 'MERN Stack',
status: 'in progress'
_id: ObjectId('6ae752963c66dea8acece031'),
name: 'Priya',
age: 21,
course: 'UI/UX Design',
status: 'enrolled'
_id: ObjectId('6ae752963c66dea8acece032'),
name: 'Rahul',
age: 24,
course: 'MERN Stack',
status: 'in progress'
_id: ObjectId('6ae752963c66dea8acece033'),
name: 'Sneha',
age: 22,
course: 'Data Analytics',
status: 'enrolled'
_id: ObjectId('6ae75ae73c66dea8acece038'),
name: 'Vidula',
age: 22,
course: 'MERN Stack'
_id: ObjectId('6ae75ba43c66dea8acece039'),
name: 'Vidula',
age: 22,
course: 'MERN Stack',
status: 'completed'
studentDB>
GIT main+
Antigravity - Settings
Go Live

4. Read Operations (Retrieve)

Read operations retrieve documents from a collection. The find() method returns all matching documents, while findOne() returns the first matching document.

4.1 Fetch all documents

```
db.students.find()
```

This command displays all documents stored in the students collection.

4.2 Fetch students enrolled in MERN Stack

```
db.students.find({
    course: "MERN Stack"
})
```

This query filters the collection and returns only students whose course field is MERN Stack.

4.3 Fetch one student

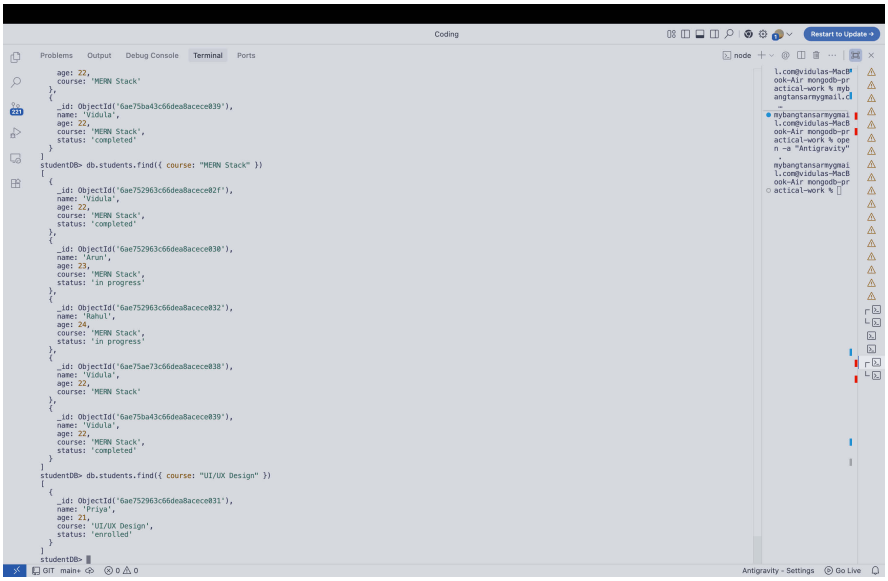
```
db.students.findOne({
    name: "Vidula"
})
```

This query returns the first document that matches the name Vidula.

4.4 Observed student records during the query-practice stage

Name	Age	Course	Status
Vidula	22	MERN Stack	completed
Arun	23	MERN Stack	in progress
Priya	21	UI/UX Design	enrolled
Rahul	24	MERN Stack	in progress
Sneha	22	Data Analytics	enrolled

The records above reflect the five-document dataset inserted for the query-operator exercises. MongoDB also generated a unique _id field for each document.



5. Update Operations

Update operations modify fields in existing documents. The $set update operator changes a field value or adds the field if it does not already exist.

5.1 Update one document

The status of the student named Vidula was changed to completed.

```
db.students.updateOne(
    { name: "Vidula" },
    { $set: { status: "completed" } }
)
```

The update operation matched one document and modified one document.

5.2 Update multiple documents

The status of all enrolled MERN Stack students was changed to in progress.

```
db.students.updateMany(
    {
        course: "MERN Stack",
        status: "enrolled"
    },
    {
        $set: { status: "in progress" }
    }
)
```

This command applies the update to every document that matches both filter conditions. During the practical, the resulting records showed Arun and Rahul with the status in progress.

SCREENSHOT PLACEHOLDER

updateOne() and updateMany() terminal results

Paste your MongoDB Shell screenshot here.

6. Delete Operations

Delete operations remove documents from a collection. deleteOne() removes the first matching document, while deleteMany() removes all documents matching the filter.

6.1 Delete one document

```
db.students.deleteOne({
    name: "Rahul"
})
```

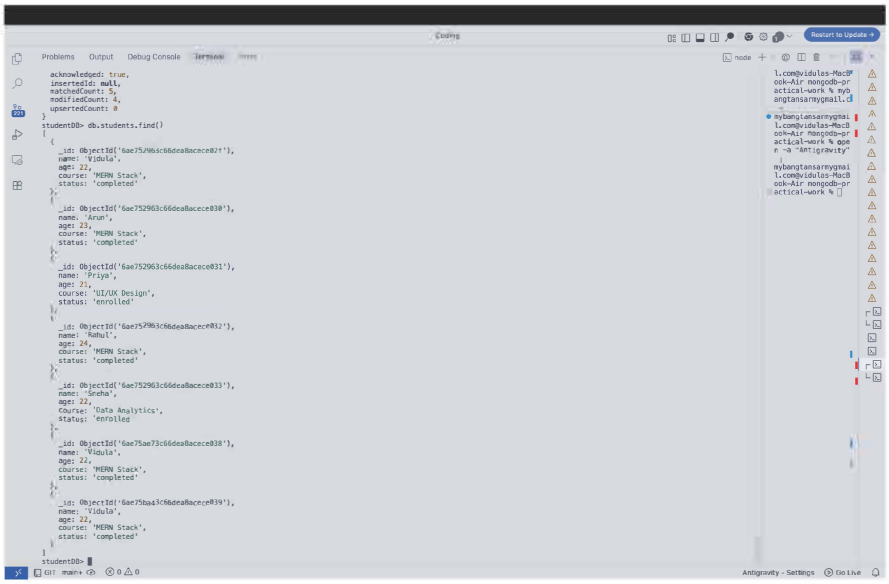
The practical output showed Rahul was no longer present in the subsequent find() result.

6.2 Delete all documents in a collection

```
db.students.deleteMany({})
```

The command returned acknowledged: true and deletedCount: 4. This means four remaining documents were deleted after Rahul had already been deleted. The collection itself remained available.

Important: deleteMany({}) removes every document in the current collection. It should only be used when intentionally clearing practice data. It does not delete the collection itself.


Problems
Output
Debug Console
Restart to Update
acknowledged: true,
insertedId: null,
matchedCount: 5,
modifiedCount: 4,
upsertedCount: 0
studentDB> db.students.find()
_id: ObjectId('6ae752963c66dea8acece030'),
name: 'Arun',
age: 23,
course: 'MERN Stack',
status: 'completed'
_id: ObjectId('6ae752963c66dea8acece031'),
name: 'Priya',
age: 21,
course: 'UI/UX Design',
status: 'enrolled'
_id: ObjectId('6ae752963c66dea8acece032'),
name: 'Rahul',
age: 24,
course: 'MERN Stack',
status: 'completed'
_id: ObjectId('6ae752963c66dea8acece033'),
name: 'Sneha',
age: 22,
course: 'Data Analytics',
status: 'enrolled'
_id: ObjectId('6ae75ae73c66dea8acece038'),
name: 'Vidula',
age: 22,
course: 'MERN Stack',
status: 'completed'
_id: ObjectId('6ae75ba43c66dea8acece039'),
name: 'Vidula',
age: 22,
course: 'MERN Stack',
status: 'completed'
studentDB>

7. MongoDB Query Operators

Query operators allow documents to be selected based on conditions such as numeric comparisons, alternative values, combined conditions and field existence.

Operator	Meaning	Purpose
$gt	Greater than	Matches values greater than a specified value.
$lt	Less than	Matches values less than a specified value.
$in	Matches any value in an array	Useful when a field can match one of several values.
$and	Logical AND	Matches documents where all specified conditions are true.
$or	Logical OR	Matches documents where at least one specified condition is true.
$exists	Field existence	Checks whether a specified field exists in a document.

7.1 $gt – Greater than

```
db.students.find({
    age: { $gt: 22 }
})
```

Observed result: Arun (age 23) and Rahul (age 24) were returned. This confirms the query matched students older than 22.

7.2 $lt – Less than

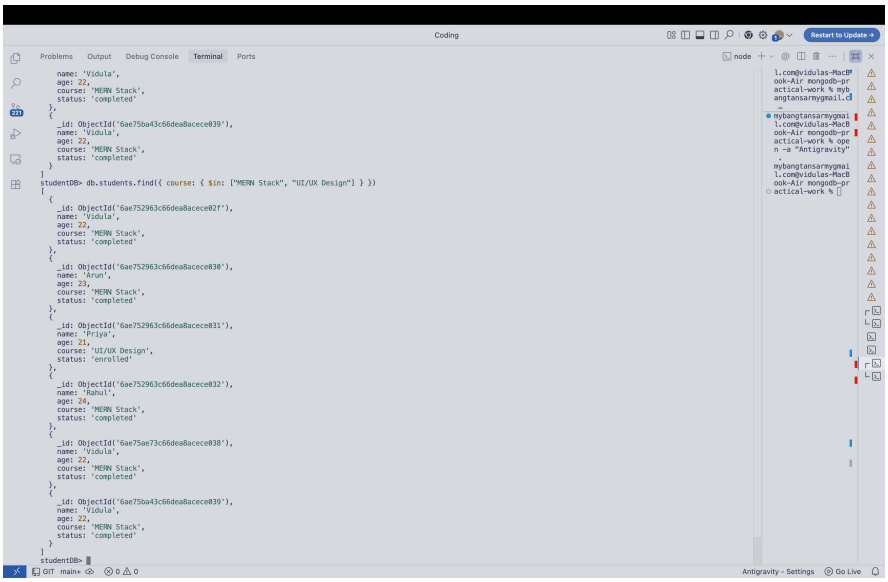
```
db.students.find({
    age: { $lt: 23 }
})
```

Observed result: Vidula (age 22), Priya (age 21) and Sneha (age 22) were returned. This confirms the query matched students younger than 23.

7.3 $in – Match multiple values

```
db.students.find({
    course: {
        $in: ["MERN Stack", "UI/UX Design"]
    }
})
```

Observed result: Vidula, Arun, Priya and Rahul were returned. Sneha was excluded because her course was Data Analytics. The query therefore matched four documents.



7. MongoDB Query Operators (continued)

7.4 $and – All conditions must be true

```
db.students.find({
    $and: [
        { age: { $gt: 21 } },
        { course: "MERN Stack" }
    ]
})
```

Verification status: NOT VERIFIED. The command and expected result are included for the assignment, but no terminal output for this query was provided. Based on the five records in the query-practice dataset, the expected matches are Vidula, Arun and Rahul.

7.5 $or – At least one condition must be true

```
db.students.find({
    $or: [
        { age: { $lt: 22 } },
        { status: "completed" }
    ]
})
```

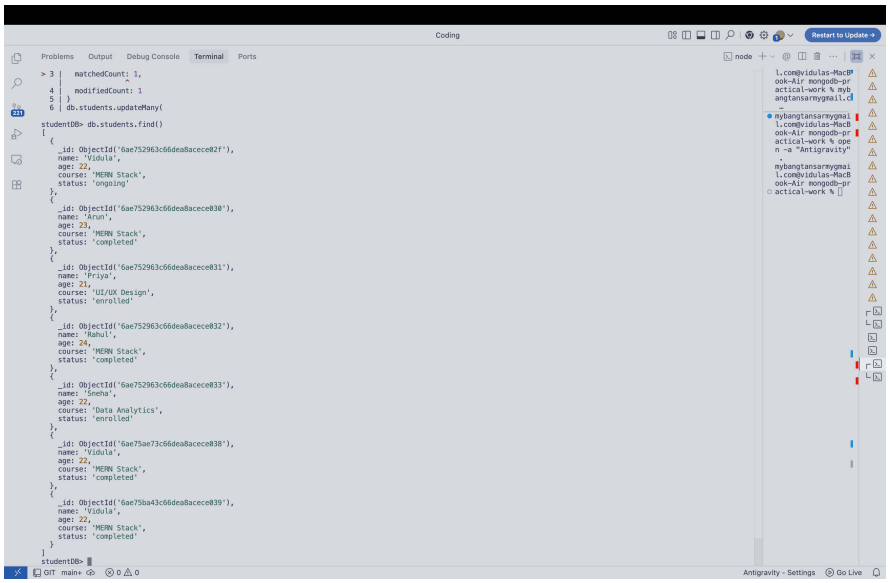
Verification status: NOT VERIFIED. No terminal output for this query was provided. Based on the five records, the expected matches are Priya (age 21) and Vidula (status completed).

7.6 $exists – Check whether a field exists

```
db.students.find({
    age: { $exists: true }
})
```

Verification status: NOT VERIFIED. No terminal output for this query was provided. Based on the five records, all five students are expected to match because each document contains an age field.

Note: The expected matches above are inferred from the dataset shown in the conversation. They should not be described as confirmed terminal results until the commands have been executed and their outputs captured.



8. Real-World Use Case: Library Management System

A Library Management System stores book information and allows a user to view books, search by availability or genre, update availability when a book is borrowed, and remove a book record when required. MongoDB can support this type of application by storing each book as a document in a books collection.

8.1 Create the database and collection

```
use libraryDB

db.createCollection("books")
```

This switches to the libraryDB database and creates a collection named books.

8.2 Insert multiple book documents

```
db.books.insertMany([
    {
        title: "Atomic Habits",
        author: "James Clear",
        genre: "Self Help",
        price: 18,
        available: true
    },
    {
        title: "The Alchemist",
        author: "Paulo Coelho",
        genre: "Fiction",
        price: 15,
        available: true
    },
    {
        title: "Rich Dad Poor Dad",
        author: "Robert Kiyosaki",
        genre: "Finance",
        price: 20,
        available: false
    },
    {
        title: "Ikigai",
        author: "Hector Garcia",
        genre: "Self Help",
        price: 16,
        available: true
    }
])
```

The terminal confirmed that four documents were inserted successfully and returned four generated ObjectIds.

8.3 Display all books

```
db.books.find()
```

The query returned all four inserted book documents.

8.4 Search for available books

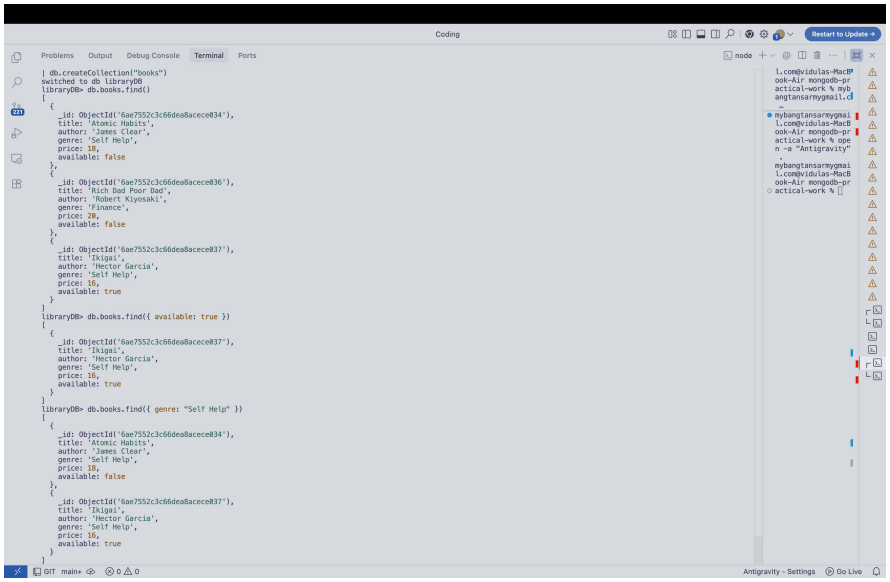
```
db.books.find({
    available: true
})
```

Observed result: Atomic Habits, The Alchemist and Ikigai were returned. Rich Dad Poor Dad was excluded because available was false.

8.5 Search books by genre

```
db.books.find({
    genre: "Self Help"
})
```

Observed result: Atomic Habits and Ikigai were returned.


Coding
Restart to Update
Problems
Output
Debug Console
Terminal
Ports
node
| db.createCollection("books")
switched to db libraryDB
libraryDB> db.books.find()
[
{
_id: ObjectId('6ae7552c3c66dea8acece034'),
title: 'Atomic Habits',
author: 'James Clear',
genre: 'Self Help',
price: 18,
available: false
},
{
_id: ObjectId('6ae7552c3c66dea8acece036'),
title: 'Rich Dad Poor Dad',
author: 'Robert Kiyosaki',
genre: 'Finance',
price: 20,
available: false
},
{
_id: ObjectId('6ae7552c3c66dea8acece037'),
title: 'Ikigai',
author: 'Hector Garcia',
genre: 'Self Help',
price: 16,
available: true
}
]
libraryDB> db.books.find({ available: true })
[
{
_id: ObjectId('6ae7552c3c66dea8acece037'),
title: 'Ikigai',
author: 'Hector Garcia',
genre: 'Self Help',
price: 16,
available: true
}
]
libraryDB> db.books.find({ genre: "Self Help" })
[
{
_id: ObjectId('6ae7552c3c66dea8acece034'),
title: 'Atomic Habits',
author: 'James Clear',
genre: 'Self Help',
price: 18,
available: false
},
{
_id: ObjectId('6ae7552c3c66dea8acece037'),
title: 'Ikigai',
author: 'Hector Garcia',
genre: 'Self Help',
price: 16,
available: true
}
]
l.com@vidulas-MacB
ook-Air mongodb-pr
actical-work % myb
angtansarmygmail.c
om
mybangtansarmygmai
l.com@vidulas-MacB
ook-Air mongodb-pr
actical-work % ope
n -a "Antigravity"
.
mybangtansarmygmai
l.com@vidulas-MacB
ook-Air mongodb-pr
actical-work %
GIT main+
Antigravity - Settings
Go Live

8. Real-World Use Case (continued)

8.6 Update book availability

When a customer borrows Atomic Habits, its availability is changed from true to false.

```
db.books.updateOne(
    { title: "Atomic Habits" },
    { $set: { available: false } }
)
```

Observed result: matchedCount was 1 and modifiedCount was 1. This confirms that one matching book document was updated.

8.7 Delete a book

```
db.books.deleteOne({
    title: "The Alchemist"
})
```

Observed result: the first delete operation returned acknowledged: true and deletedCount: 1. A second attempt returned deletedCount: 0 because the book had already been removed.

8.8 Verify the final book collection

```
db.books.find()
```

The final query returned three book documents:

Title	Genre	Price	Available
Atomic Habits	Self Help	18	false
Rich Dad Poor Dad	Finance	20	false
Ikigai	Self Help	16	true

The final records show that Atomic Habits was marked unavailable after being borrowed, The Alchemist was deleted, and the other two books remained in the collection with their existing availability values.

9. Summary of Commands

Operation	MongoDB command	Purpose
Switch database	use studentDB	Selects the database.
Create collection	db.createCollection("students")	Creates a collection.
Insert one	insertOne()	Adds one document.
Insert many	insertMany()	Adds multiple documents.
Read	find()	Retrieves matching documents.
Read one	findOne()	Retrieves the first matching document.
Update one	updateOne()	Updates one matching document.
Update many	updateMany()	Updates all matching documents.
Delete one	deleteOne()	Deletes the first matching document.
Delete many	deleteMany()	Deletes all matching documents.
Query operators	$gt, $lt, $in, $and, $or, $exists	Filters documents using conditions.

10. Conclusion

Through this assignment, I practised the fundamentals of MongoDB and learned how data is organised into databases, collections and documents. I performed CRUD operations by inserting, retrieving, updating and deleting student records. I also used query operators to filter documents based on comparison, multiple possible values, logical conditions and field existence.

The Library Management System use case demonstrated how MongoDB can be applied to a practical application. I inserted book details, searched for available books and books by genre, updated a book's availability after borrowing, and deleted a book record. These exercises provided practical experience in managing and retrieving data using MongoDB Shell.

Final evidence note: The $gt, $lt and $in outputs were supplied and verified from the practical session. The $and, $or and $exists commands are documented, but their terminal outputs remain unverified and should be run and screenshotted before submission if the instructor expects evidence for every operator.